Smart Resource Allocation Using Banker's Algorithm

Deadlock avoidance for safe, efficient resource sharing

ALLOCATION MATRIX (EXAMPLE STATE)

	Resource A	Resource B	Resource C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Operating Systems Project

Problem Statement

Concurrent processes compete for a limited pool of non-shareable resources. One careless grant can leave them waiting on each other forever: deadlock.

- A process's future demand is unknown until it declares a maximum
- Every request must be judged instantly, in real time
- Static prevention wastes resources; detection acts too late

Goal Grant a request only if the system stays in a safe state, so deadlock never occurs.

Four conditions behind every deadlock

1

Mutual exclusion

A resource can serve only one process at a time

2

Hold and wait

A process keeps resources while waiting for more

3

No preemption

Resources cannot be forcibly taken back

4

Circular wait

A cycle of processes each wait on the next

System Design

1 · INPUT

Declare claims

Processes declare their maximum claim; the pool size is set

2 · REQUEST HANDLER

Validate the request

Request ≤ Need and Request ≤ Available

3 · SAFETY ENGINE

Simulate the grant

Search for a sequence in which every process can finish

4 · DECISION

Grant or deny

Safe: commit. Unsafe: roll back and make the process wait

System state kept by the State Manager

Available

free units of each resource

Max

maximum claim per process

Allocation

units currently held

Need

Max − Allocation

Dashboard shows the matrices, the safe sequence found, and a live log of every grant and denial.

Implementation Approach

1. **Model the state**

   Arrays for Available, Max and Allocation; derive Need

2. **Build the safety check**

   Find a process whose Need fits Work, release its holdings, repeat

3. **Handle requests**

   Validate, tentatively allocate, run the safety check, commit or roll back

4. **Add the interface**

   Flask API and web dashboard to enter state and send requests

5. **Test and refine**

   Unit and scenario tests on safe, unsafe and edge cases

safety.py

```
def is_safe(avail, alloc, need):
    work, seq = avail.copy(), []
    while len(seq) < len(alloc):
        for i in range(len(alloc)):
            if i not in seq and (need[i] <= work).all():
                work += alloc[i]; seq.append(i); break
        else:
            return False, []
    return True, seq
```

Grant rule Request ≤ Need, then Request ≤ Available, then tentatively allocate. If is_safe passes, commit; otherwise roll back.

Each safety check costs $O(m \cdot n^2)$

Technologies Used

Py

Python 3

Core algorithm and simulation logic

Np

NumPy

Vector and matrix operations for Need and Work

Fl

Flask

REST API that accepts requests and returns decisions

UI

HTML · CSS · JS

Dashboard to view matrices and the safe sequence

Pt

Pytest

Automated unit and scenario tests

Git

Git and GitHub

Version control and collaboration

Testing

Unit tests

Need calculation and the safety check

Scenario tests

Classic state: 5 processes, 3 resource types, Available = (3, 3, 2)

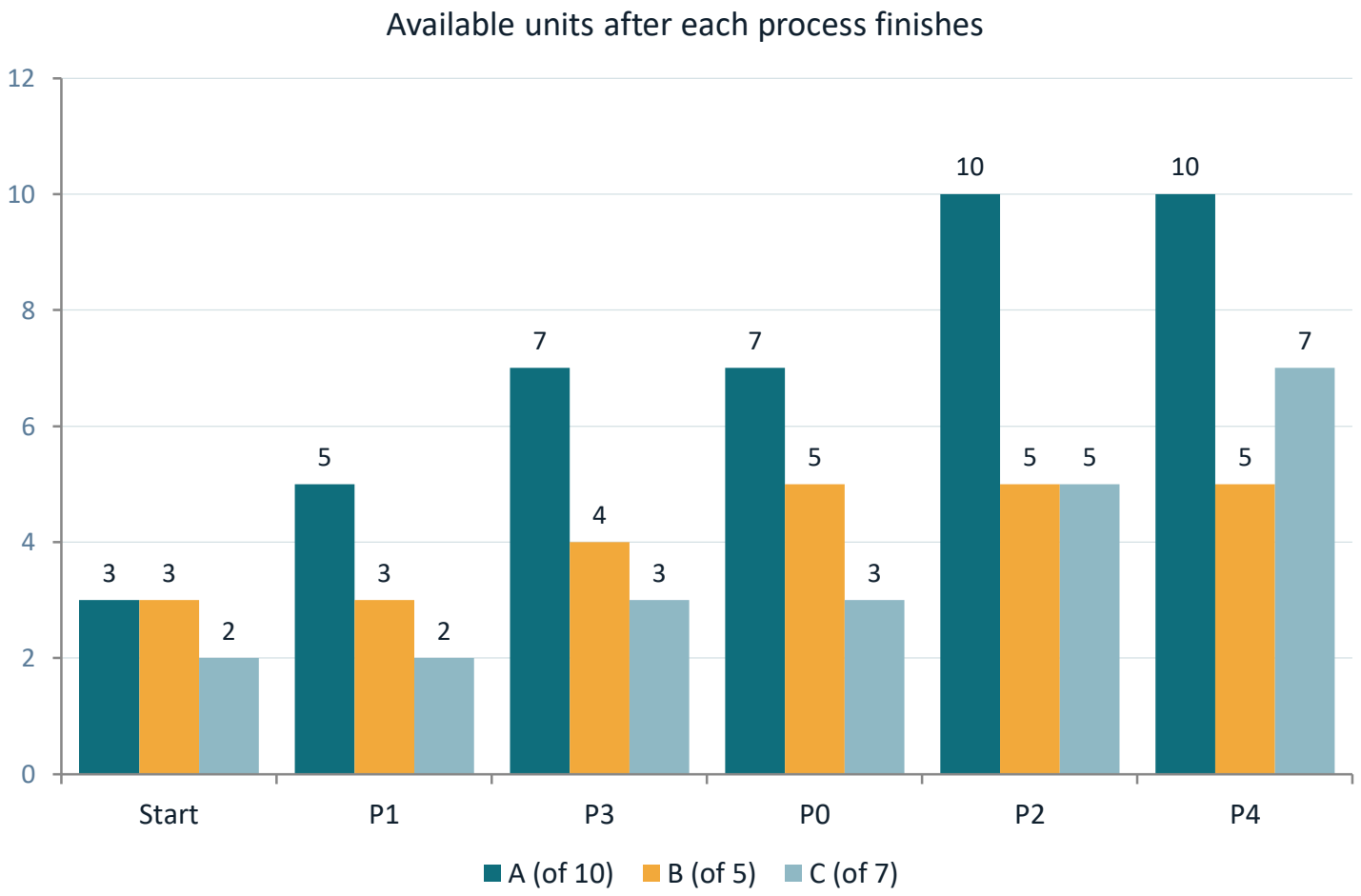
Edge cases

Over-claims, over-requests and unsafe grants

#	Scenario	Expected outcome
T1	Initial state, Available = (3, 3, 2)	Safe: P1 → P3 → P0 → P2 → P4
T2	P1 requests (1, 0, 2)	Granted; state stays safe
T3	P4 requests (3, 3, 0)	Denied: exceeds Available
T4	P0 requests (0, 2, 0)	Denied: grant would be unsafe
T5	P3 requests (1, 0, 0)	Rejected: exceeds declared Need

T2 to T4 run in sequence on the evolving state.

Results



5 / 5

processes complete in one safe sequence

0

deadlocks: the unsafe request was blocked before it was granted

O(m·n²)

time for each safety check

Impact

1

Deadlock-free by design

Unsafe grants are refused before they happen

2

Better utilisation

Resources are shared up to the safe limit, not locked up front

3

Predictable and explainable

Every decision comes with a safe sequence to show why

WHERE IT APPLIES

Operating-system schedulers

Safe handling of devices and memory

Cloud and VM resource pools

Admission control before over-commit

Database lock management

Avoiding lock cycles between transactions

Embedded and real-time systems

Predictable behaviour on scarce hardware

Trade-off: processes must declare maximum claims in advance.